Crudo

Crudo lets you take slices from raw blocks and more: slice, filter, transform and join the lines of raw blocks. *Crudo* can also extract named regions out of source files, to display them without having to hardcode linenumbers.

v0.1.1 September 28, 2024

https://github.com/SillyFreak/typst-crudo

Clemens Koza

Contents

I Introduction

`raw` elements display source code line by line, but Typst doesn't provide a lot of convenience for handling them in this way: you need to extract the `raw.text` to do anything with it, and then transform it back. But also if you read in a source file for displaying code, manipulating the resulting string as multiple lines is not too convenient. *Crudo* gives you a few extra tools to make this easier.

I.a `raw` elements as lists of lines

`raw` elements feel like lists of lines; it's common to want to extract spcific lines, join multiple ones together, etc. As values, though, `raw` elements don't behave this way.

While a package can't add methods such as `raw.slice()` to an element, we can at least provide functions to help with common tasks. The module reference describes these utility functions:

- `r2l()` and `l2r()` are the building blocks the others build on: *raw-to-lines* and *lines-to-raw* conversions.
- `transform-text()` and `transform()` are one layer above and allow arbitrarily transforming the text content, viewed as a single string or an array of strings, respectively.
- `read()` reads a text file and puts it in a raw element with the provided `raw` properties.
- `map()`, `filter()` and `slice()` are analogous to their `array` counterparts.
- `lines()` is similar to `slice()` but allows more advanced line selections in a single step.
- `join()` combines multiple `raw` elements and is convenient e.g. to add preambles to code snippets.

All functions that accept raw elements as parameters alternatively accept simple strings. In these cases, a string `code` behaves like `raw(code)`, i.e. it's not a `block` element and has no `lang` set on it. This is mostly useful with `join()`, which takes multiple raw elements, but the other functions don't disallow this usage.

I.b Handling regions in code files

Crudo also contains utilities for identifying and extracting ranges of code from source files. Let's assume you have the following source file (any language will work; we're not using Typst for the example to avoid making things too meta):

```java
// @region start:basics
public class Account {
  private double balance;

  public Konto(double balance) {
    this.balance = balance;
  }

// @region end:basics start:mutators
  public void deposit(double amount) {
    this.balance += amount;
  }

  public boolean withdraw(double amount) {
// @region start:withdraw-body
    if(this.balance - amount >= 0) {
      this.balance -= amount;
      return true;
```

```
19      }
20      return false;
21  // @region end:withdraw-body
22    }
23
24  // @region end:mutators start:basics
25    public double getBalance() {
26      return this.balance;
27    }
28  }
29  // @region end:basics
```

You can already see that this file contains a few special comments of the form `// @region ...`; these can be processed by *Crudo* to identify and extract ranges; for example, using the `regions.ranges()` function:

```
1  #let code = crudo.read("Account.java",                    typ
2    properties: (block: true, lang: "java"))
3  #crudo.regions.ranges(code, "basics") \
4  #crudo.regions.ranges(code, "mutators") \
5  #crudo.regions.ranges(code,
6    ("mutators", "!withdraw-body"))
```

```
((2, 9), (25, 29))
((10, 15), (16, 21), (22, 24))
((10, 15), (22, 24))
```

The result of `regions.ranges()` is an array of region bounds; for example, the `(2, 9)` means that lines 2 (inclusive) to 9 (exclusive) are part of the `basics` region. A leading `!` can be used to exclude a regin. Region indicators that may be embedded in a region are of course skipped, e.g. lines 15 and 21 for `withdraw-body` inside the `mutators` region.

These ranges can be further processed for use with `lines()` or packages such as codly or zebraw:

```
1  #import "@preview/codly:1.3.0"                                typ
2  #import "@preview/zebraw:0.6.1"
3  #let ranges = crudo.regions.ranges(code, "basics")
4  #grid(
5    columns: (1fr, 1fr, 1fr),
6    crudo.lines(code, ..ranges.map(((a, b)) => range(a, b))),
7    codly.codly-init[
8      #codly.codly(ranges: ranges.map(((a, b)) => (a, b - 1)))
9      #code
10   ],
11   zebraw.zebraw(line-range: ranges, code),
12 )
```

```
public class Account {
  private double balance;

  public Konto(double balance) {
    this.balance = balance;
  }

  public double getBalance() {
    return this.balance;
  }
}
```

```
2  public class Account {                                       java
3    private double balance;
4
5    public Konto(double balance) {
6      this.balance = balance;
7    }
8
25   public double getBalance() {
26     return this.balance;
27   }
28 }
```

```
                                                                java
2  public class Account {
3    private double balance;
4
5    public Konto(double balance) {
6      this.balance = balance;
7    }
8
⋮ 16 lines skipped
25   public double getBalance() {
26     return this.balance;
27   }
28 }
```

There are some differences in the exact parameters:

- `lines()` expects arrays of individual lines (which we can construct using the `range()` function). This exact usage can be simplified by using the `regions.extract()` shorthand;
- codly uses an inclusive upper bound for ranges;
- zebraw uses exclusive upper bounds and we don't need to change the ranges at all.

An important difference between the first and the latter two examples is line numbering: if we used one of the two code block libraries to format the first code snippet, we'd see line numbers from 1 to 11. This is because we are producing a new `raw` element with just a subset of the lines, and any subsequent formatting is not aware of the missing lines. In contrast, the latter two examples keep the original `raw` element, and instruct the library to only render some of those. The libraries are aware of the actual line numbers and will preserve them.

I.b.a Highlighting regions

Identifying region ranges can of course not only be used to filter lines, but also for highlighting certain ranges. Here is again an example using both codly and zebraw:

```typ
#import "@preview/codly:1.3.0"
#import "@preview/zebraw:0.6.1"
#let ranges = crudo.regions.ranges(code, "mutators")
#let highlights = crudo.regions.ranges(code, "withdraw-body")
#let highlight-lines = highlights.map(((a, b)) => range(a, b)).join()
#grid(
  columns: (1fr, 1fr),
  codly.codly-init[
    #codly.codly(ranges: ranges.map(((a, b)) => (a, b - 1)),
        highlighted-lines: highlight-lines)
    #code
  ],
  zebraw.zebraw(line-range: ranges, highlight-lines: highlight-lines, code),
)
```

```java
  public void deposit(double amount) {
    this.balance += amount;
  }

  public boolean withdraw(double amount) {
    if(this.balance - amount >= 0) {
      this.balance -= amount;
      return true;
    }
    return false;
  }

```

```java
  public void deposit(double amount) {
    this.balance += amount;
  }

  public boolean withdraw(double amount) {
⋮ 1 lines skipped
    if(this.balance - amount >= 0) {
      this.balance -= amount;
      return true;
    }
    return false;
⋮ 1 lines skipped
  }

```

As before, the displayed lines are limited to a specific region; a subregion is additionally highlighted. Both libraries take an array of line numbers for highlights. We create that by applying `range()` to each highlighted range, and joining all such ranges into a single array.

If you instead want to use the `lines()` approach (to not have gaps in the line numbers), there's a little more work to do: the highlighted range is of the numbers `(16, 21)` but the `raw` block returned by `lines()` will not contain these lines, or have different code in these lines. The `regions.ranges-within()` function can handle this situation for you:

```typ
#import "@preview/codly:1.3.0"
#import "@preview/zebraw:0.6.1"
#let ranges = crudo.regions.ranges(code, "mutators")
#let highlights = crudo.regions.ranges-within(ranges,
    crudo.regions.ranges(code, "withdraw-body"))
#let highlight-lines = highlights.map(((a, b)) => range(a, b)).join()
#codly.codly-init[
  #codly.codly(highlighted-lines: highlight-lines)
  #crudo.lines(code, ..ranges.map(((a, b)) => range(a, b)))
]
```

```java
  public void deposit(double amount) {
    this.balance += amount;
  }

  public boolean withdraw(double amount) {
    if(this.balance - amount >= 0) {
      this.balance -= amount;
      return true;
    }
    return false;
  }
```

I.c Documenting evolving code snippets

For explanatory texts it is often useful to develop a code snippet in multiple steps, where early code is later replaced by a more capable version.[1] Using the region feature shown above for that can quickly get out of hand. Consider this code as an example:

```rust
pub fn main() {
    // @region start:simple
    println!("Hello World");
    // @region end:simple start:parametric
    let name = todo!();
    println!("Hello {name}");
    // @region end:parametric start:localized
    let local_greeting = todo!();
    let name = todo!();
    println!("{local_greeting} {name}");
    // @region end:localized
}
```

This code file doesn't really consist of multiple parts that are explained separately, but different stages of development. The `main()` function declaration should always be presented, and individual parts of the implementation should be shown. To do so, the example on the next page always skips *all but one* of the implementation snippets. It gets worse when there are multiple parts of the code that evolve: the set of regions will grow, and which regions belong together becomes increasingly complex. Through this complexity, the advantage of not having to hardcode line numbers vanishes.

An additional downside is this: when executed, it would run *all parts of the logic* (the `todo!()`s notwithstanding), not just the latest one. Not even the final form of the code can be tested!

[1] In particular, this feature is inspired by *Crafting Interpreters* by Robert Nystrom. For example, in "Executing statements" right at the end, the `interpreter.interpret(statements);` line is inserted. In the accompanying source code on Github, you can see how both the old and new source code is modelled using history-aware regions.

```typ
#let code = crudo.read("greet.rs", properties: (block: true, lang: "rust"))
#grid(
  columns: (1fr, 1.2fr),
  crudo.regions.extract(code, ("!parametric", "!localized")) +
  crudo.regions.extract(code, ("!simple", "!localized")),
  crudo.regions.extract(code, ("!simple", "!parametric")),
)
```

```rust
pub fn main() {
    println!("Hello World");
}
```

```rust
pub fn main() {
    let name = todo!();
    println!("Hello {name}");
}
```

```rust
pub fn main() {
    let local_greeting = todo!();
    let name = todo!();
    println!("{local_greeting} {name}");
}
```

To handle this kind of use case, *Crudo* provides the `history` module. The `history.ranges()` and `history.extract()` functions work similar to their region counterparts, but instead of taking any number of regions, they take the single current step and a complete list of the steps in the file's history as parameters. The `history.ranges-within` function is a direct alias to `regions.ranges-within()`, as it's directly applicable to the history module as well.

Here's how to use the module for a direct comparison:

```typ
#let code = crudo.read("greet.rs", properties: (block: true, lang: "rust"))
#let history = ("simple", "parametric", "localized")
#grid(
  columns: (1fr, 1.2fr),
  crudo.history.extract(code, "simple", history) +
  crudo.history.extract(code, "parametric", history),
  crudo.history.extract(code, "localized", history),
)
```

```rust
pub fn main() {
    println!("Hello World");
}
```

```rust
pub fn main() {
    let name = todo!();
    println!("Hello {name}");
}
```

```rust
pub fn main() {
    let local_greeting = todo!();
    let name = todo!();
    println!("{local_greeting} {name}");
}
```

And here is how the code is prepared to be used with the history module:

```rust
// @start:simple
pub fn main() {
    /* @before:parametric
    println!("Hello World");
    */
    // @start:localized
    let local_greeting = todo!();
```

```
8       // @end:localized
9       // @start:parametric
10      let name = todo!();
11      /* @before:localized
12      println!("Hello {name}");
13      */
14      // @end:parametric
15      // @start:localized
16      println!("{local_greeting} {name}");
17      // @end:localized
18  }
19  // @end:simple
```

There are some differences here:

- We are not declaring regions but steps; instead of writing `@region start:` we're just writing `@start:`. Each tag can also only start or end a single step, not multiple or both, and steps must be properly nested to reflect them happening sequentially.
- The code as a whole is enclosed in a step. This is necessary: code outside a step would not be considered part of the history and wouldn't be picked up.
- There are also `@before:` tags, and these are wrapped in block comments. That means when running the annotated code, only the final version of the code is actually used. A `@before` tag's content will be removed from the enclosing step when the declared step is reached. For example, line 12 in the `@before:localized` tag will be displayed in the `parametric` step, but not later.

You may have also noticed that the `let name = todo!();` line is no longer duplicated; achieving that with regions would have been way harder.

That `@before` tags are block comments also has its downsides, unfortunately: when not using `extract` and instead using a code block library to limit the lines, the syntax highlighting will pick up on the code actually being a comment:

```typ
1  #let ranges = crudo.history.ranges(
2      code, "simple", history)
3  #codly.codly(ranges: ranges.map(
4      ((a, b)) => (a, b - 1)))
5  #code
```

```rust
2   pub fn main() {
4       println!("Hello World");
18  }
```

The history module is therefore less useful for this kind of usage.

II Module reference

`r2l(raw-block: content str) -> array`

raw-to-lines: extract lines and properties from a `raw` element.

```typc
crudo.r2l(```txt
first line
second line
```)
```

```
(
  ("first line", "second line"),
  (block: true, lang: "txt"),
)
```

Note that even though you will usually want to use this on raw *blocks*, this is not a necessity:

```typc
crudo.r2l(
  raw("first line\nsecond line")
)
```

```
(("first line", "second line"), (:))
```

For flexibility, regular strings are also supported. Strings don't have a language and aren't blocks:

```typc
crudo.r2l("first line\nsecond line")
```

```
(("first line", "second line"), (:))
```

Parameters:

`raw-block` (`content` or `str`) – a single `raw` element or (multi line) string

---

`l2r(lines: array, ..properties: arguments) -> content`

lines-to-raw: convert lines into a `raw` element. Properties for the created element can be passed as parameters.

```typc
crudo.l2r(
  ("first line", "second line")
)
```

```
first line
second line
```

Note that even though you will usually want to construct raw *blocks*, this is not assumed. To create blocks, pass the appropriate parameter:

```typc
crudo.l2r(
  ("first line", "second line"),
  block: true,
)
```

```
  first line
  second line
```

Parameters:

`lines` (`array`) – an array of strings

`..properties` (`arguments`) – properties for constructing the new `raw` element

`transform-text(raw-block: content str, mapper: function) -> content`

Transforms the text of a raw element and creates a new one with the new text. All properties of the element (e.g. `block` and `lang`) are preserved.
```

```typc
crudo.transform-text(
  ```typc

  let foo() = {
    // some comment
    ... do something ...
  }
  ```,
  str.trim
)
```

```
let foo() = {
  // some comment
  ... do something ...
}
```

Parameters:

`raw-block` (`content` or `str`) – a single `raw` element or (multi line) string

`mapper` (`function`) – a function that takes a single string and returns a new one

`transform(raw-block: content str, mapper: function) -> content`

Transforms all lines of a raw element and creates a new one with the lines. All properties of the element (e.g. `block` and `lang`) are preserved.

```typc
crudo.transform(
  ```typc
  let foo() = {
    // some comment
    ... do something ...
  }
  ```,
  lines => lines.filter(l => {
    // only preserve non-comment lines
    not l.starts-with(regex("\s*//"))
  })
)
```

```
let foo() = {
  ... do something ...
}
```

Parameters:

`raw-block` (`content` or `str`) – a single `raw` element or (multi line) string

`mapper` (`function`) – a function that takes an array of strings and returns a new one

`read(properties: dictionary, trim: boolean alignment, ..args: arguments) -> content`

A wrapper around the built-in `read()` function that returns the file contents as a raw element. Since code files often have a trailing newline by convention, this function can optionally trim the file contents (and trims the end by default).

Parameters:

`properties` (`dictionary` = `(:)`) – properties for constructing the new `raw` element, given as a dictionary instead of direct arguments since the latter is used for the `read()` parameters

`trim` (`boolean` or `alignment` = `end`) – one of `true`, `false`, `start`, `end` to determine whether and what to `trim()` from the read file

`..args` (`arguments`) – the parameters to `read()`, i.e. file name and encoding

`map(raw-block: content str, mapper: function) -> content`

Maps individual lines of a raw element and creates a new one with the lines. All properties of the element (e.g. `block` and `lang`) are preserved.

````
crudo.map(                                typc
  ```typc
  let foo() = {
    // some comment
    ... do something ...
  }
  ```,
  l => l.trim()
)
````

```
let foo() = {
// some comment
... do something ...
}
```

Parameters:

`raw-block` (`content` or `str`) – a single `raw` element or (multi line) string

`mapper` (`function`) – a function that takes a string and returns a new one

`filter(raw-block: content str, test: function) -> content`

Filters lines of a raw element and creates a new one with the lines. All properties of the element (e.g. `block` and `lang`) are preserved.

````
crudo.filter(                             typc
  ```typc
  let foo() = {
    // some comment
    ... do something ...
  }
  ```,
  l => not l.starts-with(regex("\s*//"))
)
````

```
let foo() = {
  ... do something ...
}
```

Parameters:

`raw-block` (`content` or `str`) – a single `raw` element or (multi line) string

`test` (`function`) – a function that takes a string and returns a new one

`slice(raw-block: content str, ..args: arguments) -> content`

Slices lines of a raw element and creates a new one with the lines. All properties of the element (e.g. `block` and `lang`) are preserved.

```typc
crudo.slice(
  ```typc
  let foo() = {
    // some comment
    ... do something ...
  }
  ```,
  1, 3,
)
```

```
// some comment
... do something ...
```

Parameters:

`raw-block` (`content` or `str`) – a single `raw` element or (multi line) string

`..args` (`arguments`) – the same arguments as accepted by `array.slice()`

`lines(raw-block: content str, ..line-numbers: arguments, zero-based: boolean) -> content`

Extracts lines of a raw element similar to how e.g. printers select page ranges. All properties of the element (e.g. `block` and `lang`) are preserved.

This function is comparable to `slice()` but doesn't have the the option to specify the *number* of selected lines via `count`. On the other hand, multiple ranges of pages can be selected, and indices are one-based by default, which may be more natural for line numbers.

Lines are selected by any number of parameters. Each parameter can take either of three forms:

- a single number: that line is included in the output
- an array of numbers: these lines are included in the output (a major usecase being `range()` – but beware that `range()` uses an exclusive end index)
- a string containing numbers (e.g. `"1"`) and inclusive ranges (e.g. `"2-3"`) separated by commas. Range limits may be omitted (e.g. `"-2"`, `"2-"`), meaning the range starts/ends at the first/last line. Whitespace is allowed.

All three kinds of parameters can be mixed, and lines can be selected any number of times and in any order.

```typc
crudo.lines(
  ```typc
  let foo() = {
    // some comment
    ... do something ...
    // another comment
  }
  ```,
  "-2,4-,1", "2-3", range(3, 5), 5,
)
```

```
let foo() = {
  // some comment
  // another comment
}
let foo() = {
  // some comment
  ... do something ...
  ... do something ...
  // another comment
}
```

Parameters:

`raw-block` (`content` or `str`) – a single `raw` element or (multi line) string

`..line-numbers` (`arguments`) – any number of line number specifiers, as described above

`zero-based` (`boolean` = `false`) – whether the supplied numbers are one-based line numbers or zero-based indices

`join(..raw-blocks, main) -> content`

Joins lines of multiple raw elements and creates a new one with the lines. All properties of the `main` element (e.g. `block` and `lang`) are preserved.

```typc
crudo.join(
  ```java
  let foo() = {
    // some comment
    ... do something ...
  }
  ```,
  ```typc
  let bar() = {
    // some comment
    ... do something ...
  }
  ```,
  main: -1,
)
```

```
let foo() = {
  // some comment
  ... do something ...
}
let bar() = {
  // some comment
  ... do something ...
}
```

String parameters are allowed; the `main` parameter defaults to the first raw block:

```typc
crudo.join(
  "// these strings don't",
  "// determine the properties",
  ```typ
  // this raw block does:
  // still Typst!
  ```,
)
```

```
// these strings don't
// determine the properties
// this raw block does:
// still Typst!
```

- ..raw-blocks (arguments): any number of single `raw` elements or (multi line) strings
- main (int, auto): the index of the `raw` element of which properties should be preserved. Negative indices count from the back. `auto` chooses the first positional argument that is a `raw` element and not a string, if any.

II.a `regions`

- `ranges()`
- `extract()`
- `ranges-within()`

`ranges(raw-block: content str, regions: none str array) -> array`

Returns an array of line ranges; each range is a pair of numbers representing the lower (inclusive) and upper (exclusive) bound of the range. There is no one to one correspondence between the requested `regions` and the returned ranges:

- if a region contains region markers, those markers will be excluded, resulting in multiple ranges for that region;
- if a region appears multiple times in the `raw-block`, each appearance will have its own range(s).

The resulting ranges can be used with `lines()` (although see `extract()` as a shortcut), `ranges-within()`, or with libraries such as codly or zebraw. See Section I.b for more examples on using regions.

Parameters:

`raw-block` (`content` or `str`) – a single `raw` element or (multi line) string

`regions` (`none` or `str` or `array`) – a single or array of region names, or `none` to get all lines that don't indicate regions. Prefixing a region with `!` excludes the region instead of excluding it.

`extract(raw-block: content str, regions: none str array) -> content`

Uses `ranges()` combined with `lines()` to select a subset of lines from a code snippet. This function is equivalent to

```
1 lines(raw-block, ..ranges(raw-block, regions).map(((a, b)) => range(a, b)))    typc
```

Parameters:

`raw-block` (`content` or `str`) – a single `raw` element or (multi line) string

`regions` (`none` or `str` or `array`) – a single or array of region names, or `none` to get all lines that don't indicate regions. Prefixing a region with `!` excludes the region instead of excluding it.

`ranges-within(outer-ranges: array, inner-ranges: array) -> array`

Translates the `inner-ranges` line numbers for use in a code block that has been reduced to only contain the ranges from `outer-ranges`.

Let's say we have a code snippet with 20 lines and wanted to skip lines 1-2 and 11-12. We would therefore specify `outer-ranges` as `((3, 11), (13, 21))` (upper bound is exclusive). In the resulting

snippet, we want to refer to lines 4 and 14-15, so we specify `inner-ranges` as `((4, 5), (14, 16))` —but because we're removing lines, we need to use different line numbers to refer to these! `ranges-within()` will give us the correct result ranges of `((2, 3), (10, 12))`.

If the `inner-ranges` contain lines that are not contained in the `outer-ranges`, these will be dropped. See Section I.b.a for more examples on using ranges within ranges.

Parameters:

`outer-ranges` (`array`) – an array of line ranges, as returned by `ranges()`. This determines what lines are present in the resulting code snippet and what line numbers are skipped.

`inner-ranges` (`array`) – an array of line ranges, as returned by `ranges()`. This determines what lines should be selected within the outer ranges. Lines not inside the outer ranges are skipped, and line numbers are changed to skip any lines that don't appear in the outer ranges.

II.b `history`

- `ranges()`
- `extract()`
- `ranges-within`

`ranges(raw-block: content str, current: str, history: array) -> array`

Returns an array of line ranges; each range is a pair of numbers representing the lower (inclusive) and upper (exclusive) bound of the range. The ranges that are returned are specified via the `current` step, which must be an element of the specified `history`. The lines that are returned are those that come in the history before or at `current`, but have not been removed through an `@before` tag.

The resulting ranges can be used with `lines()` (although see `extract()` as a shortcut), `ranges-within`, or with libraries such as codly or zebraw. See Section I.c for more examples on using histories.

Parameters:

`raw-block` (`content` or `str`) – a single `raw` element or (multi line) string

`current` (`str`) – the step at which to show the file

`history` (`array`) – an array of step names that appear in the file, establishing the order of modifications to determine what steps have come before `current`.

`extract(raw-block: content str, current: str, history: array) -> content`

Uses `ranges()` combined with `lines()` to select a subset of lines from a code snippet. This function is equivalent to

```
lines(raw-block, ..ranges(raw-block, current, history).map(((a, b)) =>
range(a, b)))
```
typc

Parameters:

`raw-block` (`content` or `str`) – a single `raw` element or (multi line) string

`current` (`str`) – the step at which to show the file

`history` (`array`) – an array of step names that appear in the file, establishing the order of modifications to determine what steps have come before `current`.

`ranges-within`: `function`

This is a re-export of `regions.ranges-within()` since that function is also appropriate for the ranges produced by this module.